สร้าง RESTful API ด้วยภาษา TypeScript / JavaScript

ปูพื้นฐานฉบับรวบรัด
เหมาะสำหรับคนรีบไปกินข้าวเย็น
Week 2



Frontend using React

Using Vite and Typescript

What’s React?

- React/React.js คือ Library สำหรับช่วยพัฒนาเว็บไซต์ในภาษา Javascript ให้รวดเร็วและง่ายยิ่งขึ้น
  - เน้นสำหรับการสร้าง UI และ Interaction กับผู้ใช้
- สามารถสร้าง UI Component ได้ง่าย
- เขียน UI ด้วย JSX (**J**ava**S**cript **X**ML) โดยใช้ ***HTML Tags***
- ~~มีการใช้ Virtual DOM ในขั้นตอนการ Render~~ *ไม่ต้องรู้ก็ได้
- มักใช้ร่วมกับ Compiler/Transpiler และ Bundler เช่น Bable, Vite, Webpack

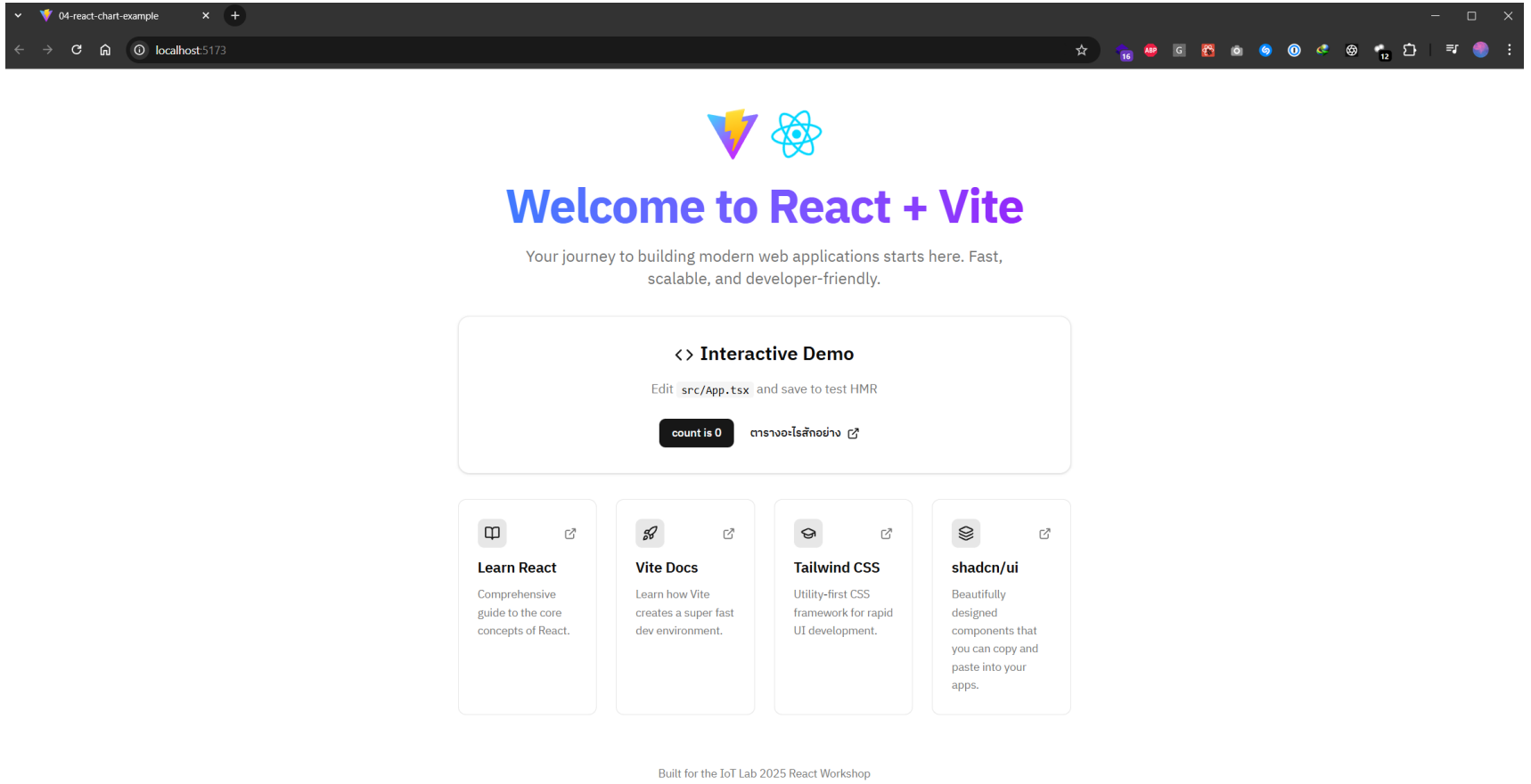
What’s Vite? *(อ่านว่า วี๊ด)*

- Vite is “Build” tool สำหรับพัฒนาเว็บไซต์สมัยใหม่ รองรับหลาย Library/Framework เช่น
  - React
  - Vue
  - Svelte
  - อื่น ๆ อีกหลายตัว
- มี Built-In Compiler/Transpiler ในตัวพร้อม Bundler
- มีเครื่องมืออื่น ๆ ช่วยพัฒนาเว็บ Built-In มาให้เลย เช่น Development Server, HMR พร้อมเครื่องมือ Integration กับ IDE อื่น ๆ เป็นต้น
- รองรับการทำ SSR (Server-Side Rendering)
- รองรับภาษา Typescript

Setup Frontend Project

- Install Node.js 24LTS
- Pull Update จาก Repository
- Cd to Frontend Project – ตัวอย่างที่ 4
- Install Project Depedencies “npm install”
- Copy “.env.example” to “.env”
- ตั้งค่าในไฟล์ “.env”
- Run “npm run dev” to start Development Server
- เปิดเว็บไซต์ https://localhost:5173

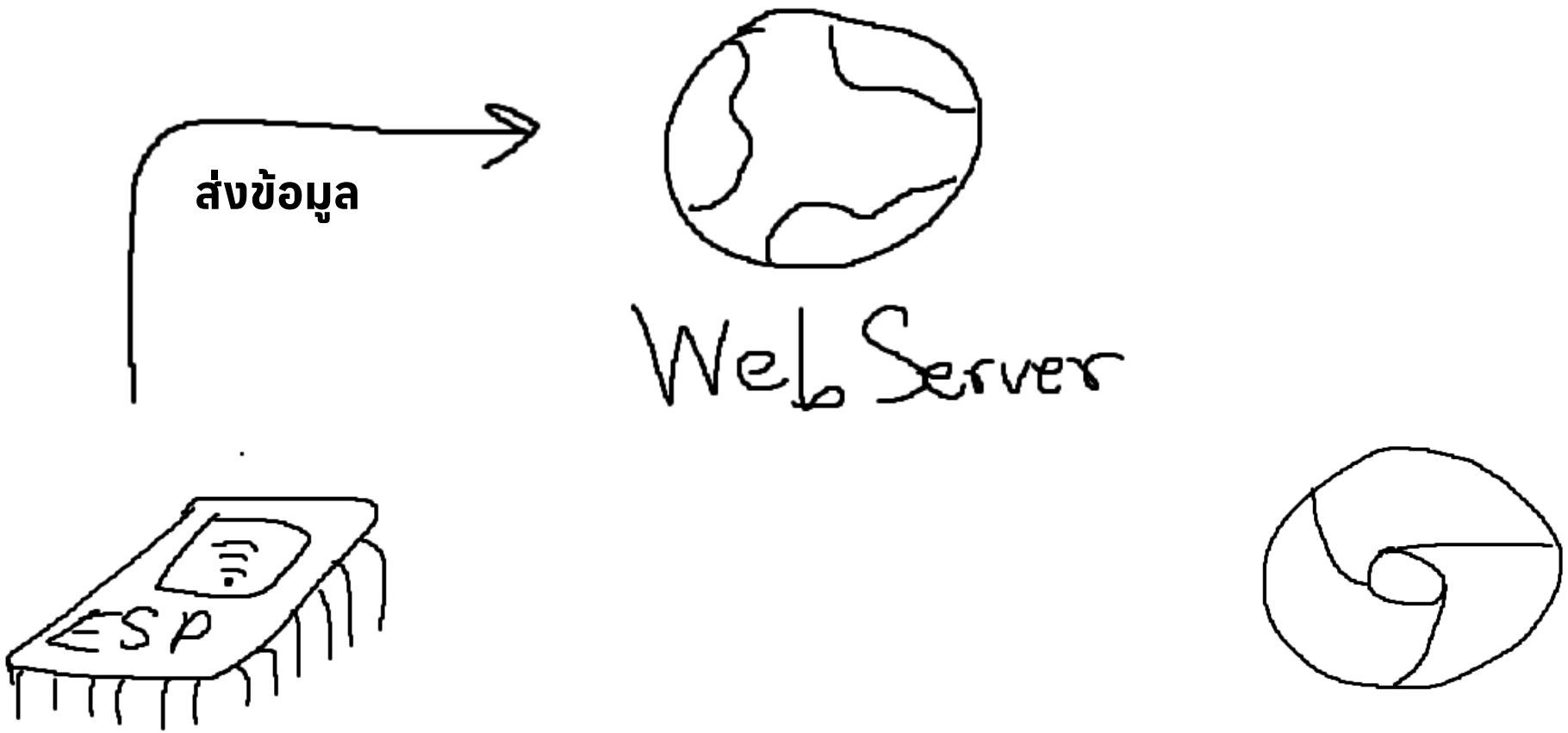
View Frontend Project



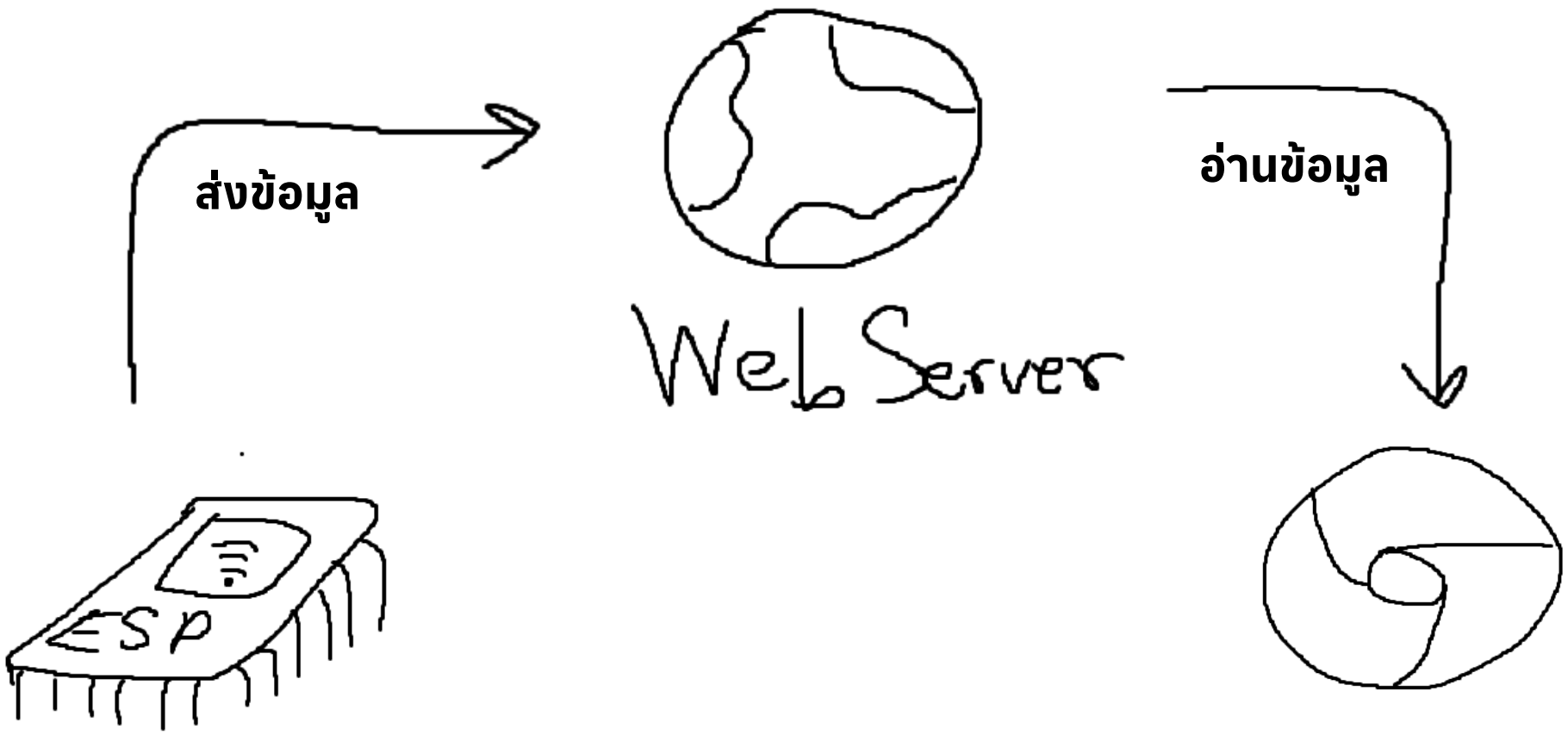
Example IoT & API Structure Diagram



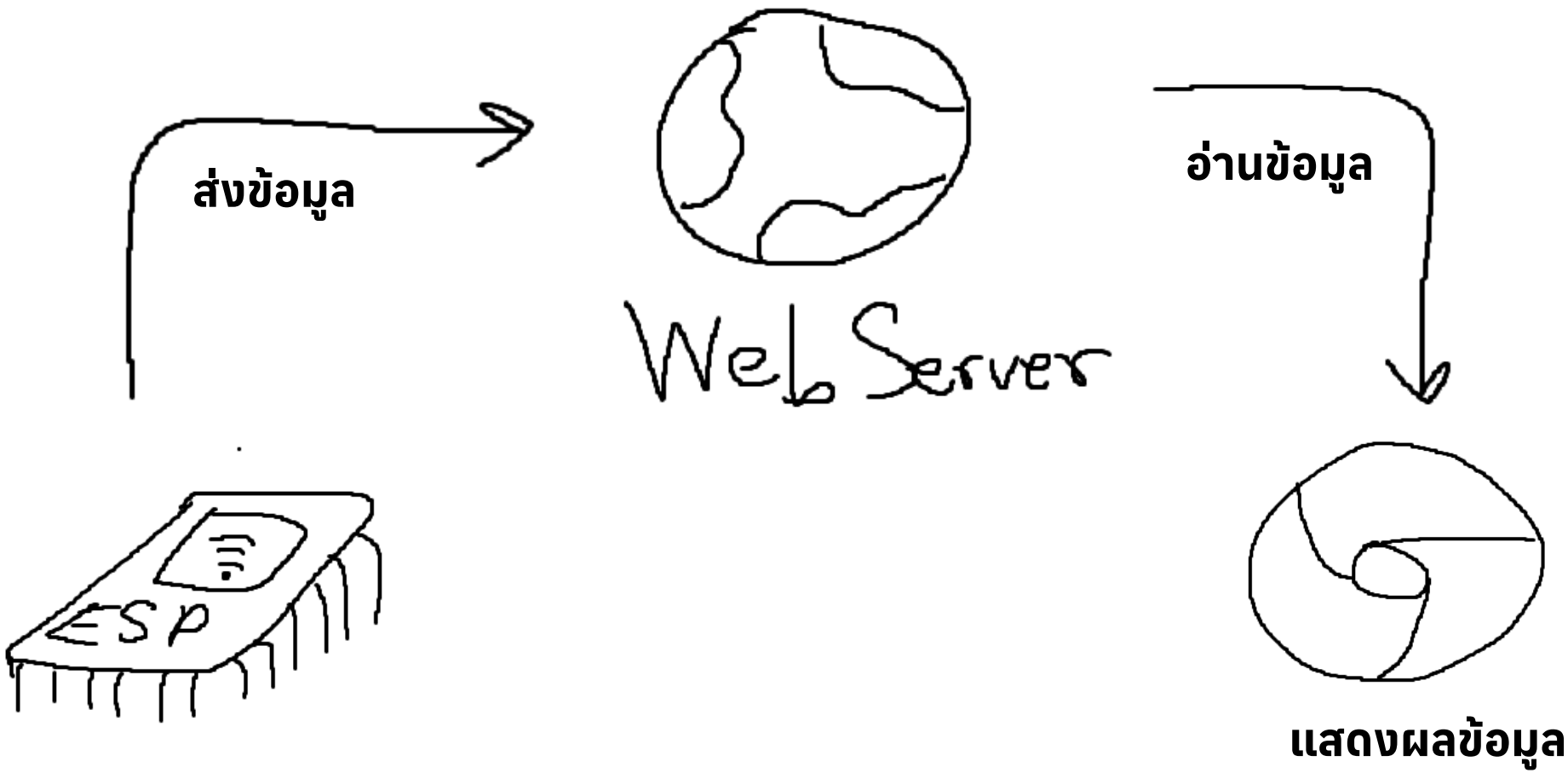
Example IoT & API Structure Diagram



Example IoT & API Structure Diagram



Example IoT & API Structure Diagram



Example Web Service with Data from Sensor

- Using Code from **Example 5** included
  - **API** for Web Service
  - **Web App** for Website Application running On Web Browser using React
- **Setup** API and Web App “.env”
- Run Database Migration
- Try adding data to Database

WebSocket

Web Communication Protocol

How to Get Real-time data from server?

Or mostly real-time

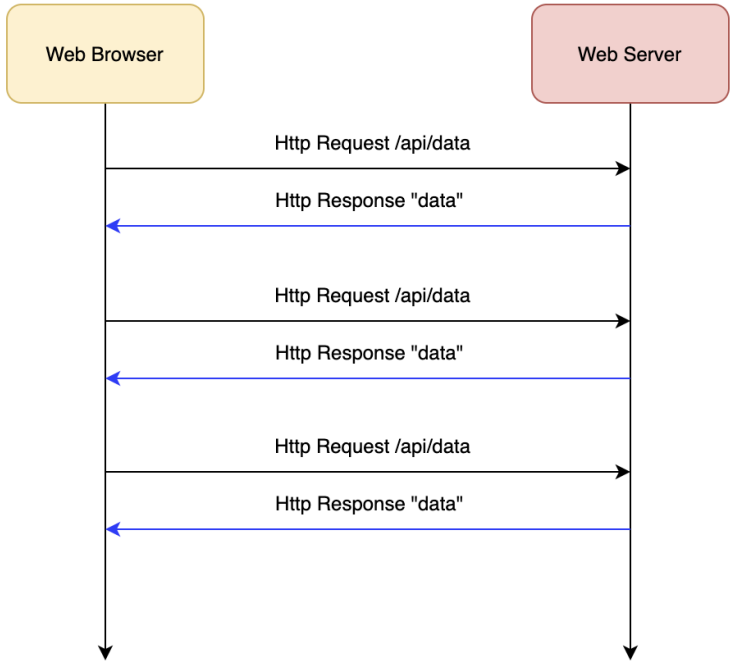
- Polling – Short and Long Polling
  - Client ร้องขอข้อมูลไปยัง Server เป็นระยะๆ
    - Short Polling ขอไป ไม่มีก็บอกไม่มี เดี๋ยวสักระยะเวลาหนึ่งก็จะขอใหม่
    - Long Polling ขอไป ไม่มีก็คง Connection ค้างไว้ก่อน ถ้ามีข้อมูลมาก็รับมา ไม่มีก็รอไปเรื่อยๆ *แต่ถ้า Timeout ก็ขอใหม่ รอใหม่*
- Pushing
  - Server ส่งข้อมูลกลับมา ระหว่างที่ Client Request ไป **ส่งสิ่งที่ Client ไม่ได้ขอแต่จำเป็น หรือต้องการส่งกลับมาให้**
- WebSocket

What’s WebSocket?

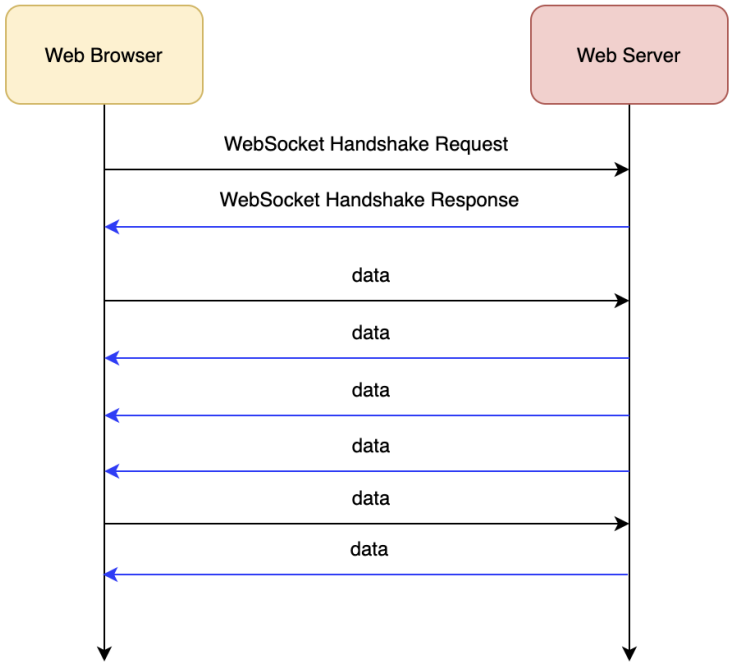
- เป็น Protocol สำหรับการเชื่อมต่อบน Web Service
- รองรับการรับส่งข้อมูลแบบ Full-Duplex
- นิยมใช้ในการส่งข้อมูลแบบ Real-Time
- มีการเปิดการเชื่อมต่อ และคงการเชื่อมต่อ TCP ไว้ระหว่างการติดต่อสื่อสาร
- Request เริ่มต้นมักเริ่มด้วย HTTP และจะมีการส่ง HTTP Upgrade ในขั้นตอนของการทำ HandShake ไปเป็น WebSocket
- ใช้ พร:// หรือ พรร:// นำหน้าใน URI

WebSocket vs HTTP

HTTP Request (HTTP 1.1)



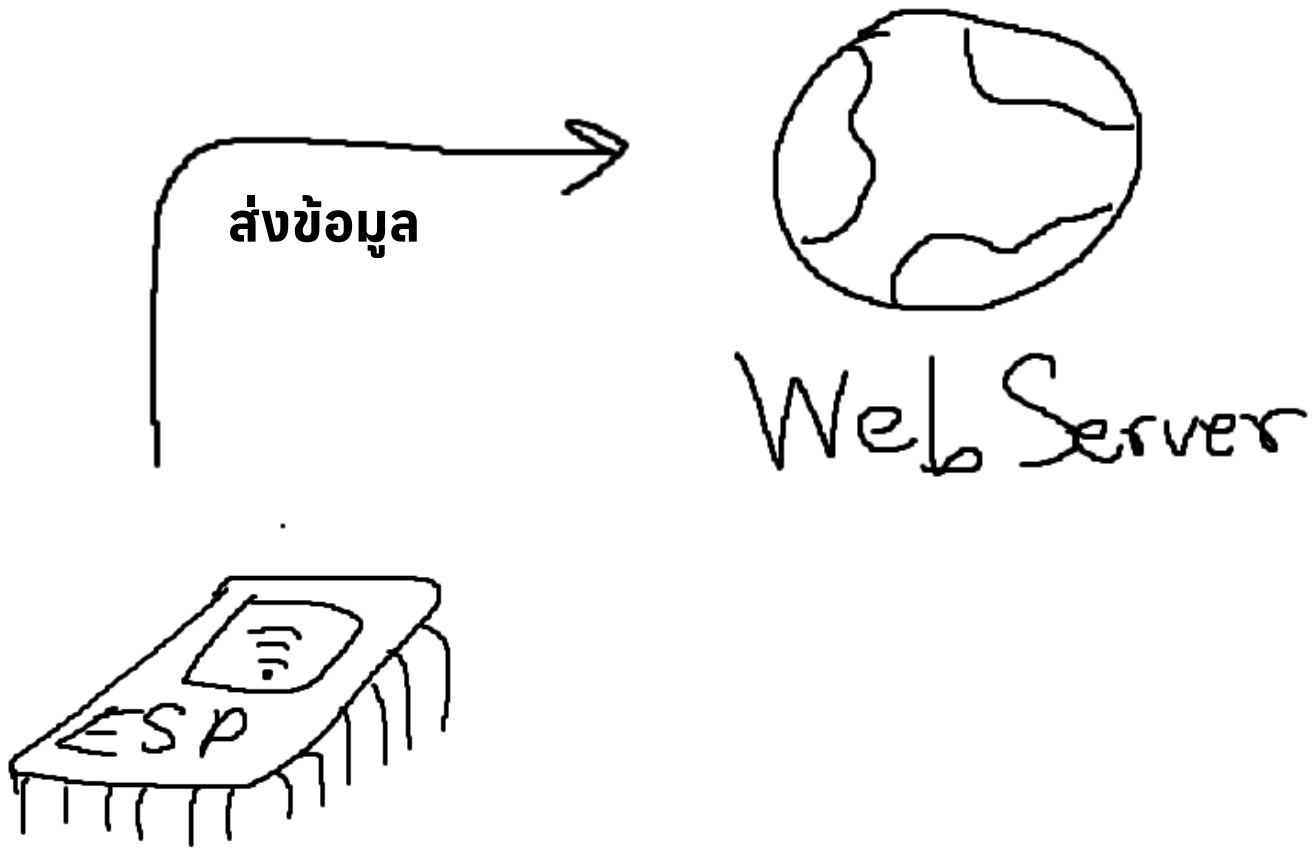
Web Socket



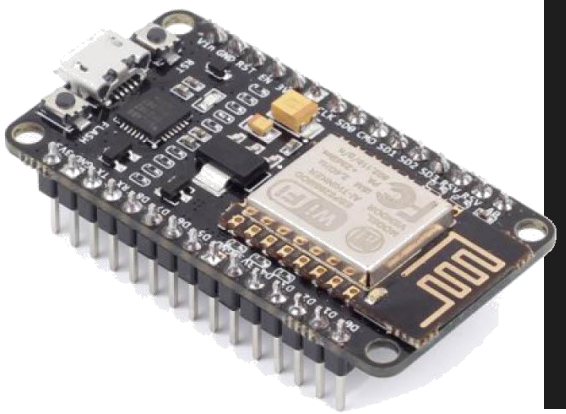
Example Web Service with Data from Sensor

- Using Code from **Example 5.**
  - **API** for Web Service
  - **Web App** for Website Application running On Web Browser using React
- **Setup** API and Web App “.env”
- Run Database Migration
- Try adding data to Database
- **Use WebSocket or Polling to get (near)real-time data from Web Server**

How to send data from Sensor?



Node MCU



```ino
#include <ESP8266WiFi.h>
#include <ESP8266HTTPClient.h>
#include <WiFiClient.h>

const char* ssid = "YOUR_SSID";
const char* pass = "YOUR_PASS";
const char* url = "http://192.168.1.51:3000/";

void setup() {
  Serial.begin(115200);
  WiFi.begin(ssid, pass);
  while (WiFi.status() != WL_CONNECTED) delay(300);

  WiFiClient client;
  HTTPClient http;

  http.begin(client, url);
  http.addHeader("Content-Type", "application/json");

  String body =
    "{\"sensorName\":\"Freezer
Temp\",\"sensorType\":\"Temperature\","
    "\"value\":10.23,\"unit\":\"c\"}";

  int code = http.POST(body);
  Serial.println(code);
  Serial.println(http.getString());
  http.end();
}

void loop() {}
```

```ino
float readTempC() {
  const float R_FIXED = 10000.0f;      // 10k series resistor
  const float BETA = 3950.0f;          // common NTC beta
  const float T0 = 298.15f;            // 25C in Kelvin
  const float R0 = 10000.0f;           // 10k at 25C
  const float ADC_MAX = 1023.0f;

  // Average a few samples for stability
  uint32_t sum = 0;
  for (int i = 0; i < 10; i++) {
    sum += analogRead(A0);
    delay(5);
  }
  float adc = sum / 10.0f;

  // Avoid divide-by-zero / out-of-range
  if (adc <= 1.0f) adc = 1.0f;
  if (adc >= ADC_MAX - 1.0f) adc = ADC_MAX - 1.0f;

  // Voltage divider math -> NTC resistance
  float rNtc = R_FIXED * (adc / (ADC_MAX - adc));

  // Beta equation -> Kelvin
  float invT = (1.0f / T0) + (1.0f / BETA) * log(rNtc / R0);
  float tempK = 1.0f / invT;

  return tempK - 273.15f; // C
}

void postTemp(float tempC) {
  WiFiClient client;
  HTTPClient http;

  http.begin(client, url);
  http.addHeader("Content-Type", "application/json");

  String body =
    String("{\"sensorName\":\"Freezer
Temp\",\"sensorType\":\"Temperature\",") +
    "\"value\":" + String(tempC, 2) + ",\"unit\":\"c\"}";

  http.POST(body);
  http.end();
}
```

Raspberry Pi



```python
def read_temp_c():
    base = "/sys/bus/w1/devices/"
    devs = glob.glob(base + "28-*")
    if not devs:
        raise RuntimeError("No DS18B20 found (28-*)")
    path = devs[0] + "/w1_slave"

    # retry until CRC OK
    for _ in range(5):
        with open(path, "r") as f:
            lines = f.read().strip().splitlines()

        if lines[0].endswith("YES"):
            tpos = lines[1].find("t=")
            if tpos != -1:
                return float(lines[1][tpos + 2 :]) / 1000.0
        time.sleep(0.2)

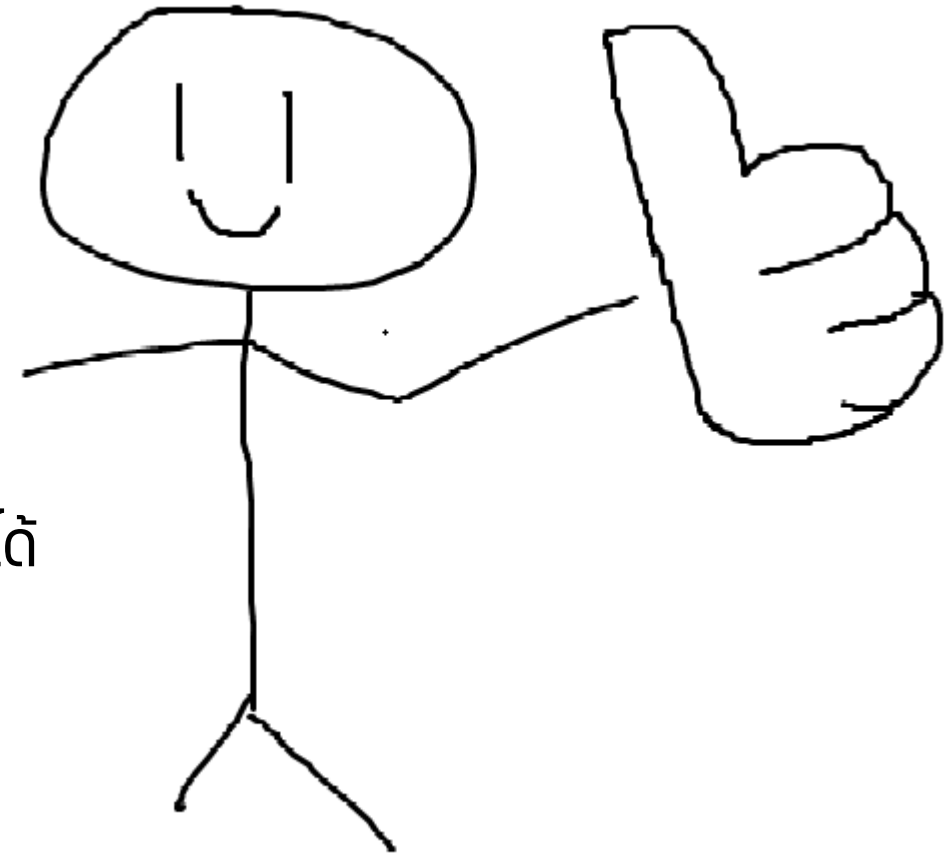
    raise RuntimeError("Bad DS18B20 read (CRC not OK)")

def post_temp(temp_c):
    payload = {
        "sensorName": "Freezer Temp",
        "sensorType": "Temperature",
        "value": round(temp_c, 2),
        "unit": "c",
    }
    r = requests.post(URL, json=payload, timeout=5)
    print("HTTP", r.status_code, r.text)
```

MQTT Will Return in Avenger Doomsday.

Thank you!

หากมีคำถาม สามารถสอบถามได้



Krittanut Siripornnoppakhun

Facebook / Gubhit / LinkedIn